SQL as a Programming Language

Implementing Distance-Vector Routing with Recursive CTEs in DuckDB

Mehdi Hoseyni • Database Systems Seminar • University of Tubingen

Summer Semester 2026

AGENDA

MOTIVATION AND RESEARCH QUESTION

Research Question

- **Core Question:** Can SQL serve as a first-class programming language for complex algorithmic simulation?
- **Approach:** Implement the Distance-Vector Routing protocol entirely within SQL, using DuckDB's recursive CTEs.
- **Goal:** Prove that declarative logic with state-aware recursion can achieve global optimization through local updates.

Why It Matters

- **Traditional View:** SQL is seen as a data retrieval language, not suitable for algorithmic modeling.
- **Our Claim:** Modern SQL extensions (recursive CTEs, USING KEY) enable full simulation of distributed systems.
- **Implication:** Database engines can function as high-level environments for protocol validation.

Thesis Statement: Modern SQL, specifically through state-aware recursive CTEs, provides a robust framework for simulating complex distributed algorithms. DuckDB's `USING KEY` syntax allows a 1:1 conceptual mapping to physical router behavior.

DISTANCE-VECTOR ROUTING THEORY

The Protocol

- **Distributed Algorithm:** Each router maintains a distance vector of costs to all destinations.
- **Local Knowledge Only:** Routers share tables only with immediate neighbors.
- **Iterative Convergence:** Updates repeat until all routers agree on optimal paths.
- **"Routing by Rumor":** Each router trusts its neighbor's advertised distances.

$$d(u, v) = min_{w \in Adj(u)} \{ cost(u, w) + d(w, v) \}$$

The Bellman-Ford relaxation equation

Convergence Process

PHASE	ACTION
T = 0	Each router knows direct neighbors only
T = 1	Share tables with neighbors, update if cheaper path found
T = 2	Repeat relaxation; discover indirect paths
T = k	Convergence: no more improvements possible

Guarantee: Converges in at most |V| - 1 rounds for a graph with |V| vertices (no negative cycles).

SQL RECURSIVE CTES: THE CHALLENGE

Standard CTE Problem

- **Append-Only Semantics:** Standard SQL recursive CTEs can only add new rows, never update existing ones.
- **Path Explosion:** Every possible walk is enumerated, leading to exponential row growth in dense graphs.
- **Post-Hoc Aggregation:** A final `GROUP BY ... MIN(cost)` is required to extract optimal paths.
- **Scalability Limit:** Networks beyond ~12 nodes cause SQLite to exhaust disk/memory.

What Routing Needs

- **In-Place Updates:** Routers overwrite their tables when a better route arrives.
- **Bounded Working Set:** Only $O(V^2)$ entries should exist at any time.
- **Automatic Termination:** Stop when no improvement is possible, not after a fixed iteration count.
- **Key-Based Pruning:** Maintain only the best cost per (source, destination) pair.

Core Mismatch: Standard SQL recursion builds a growing set of paths. DVR requires a fixed-size table that is refined iteratively. This is the fundamental tension this work resolves.

DUCKDB USING KEY: THE INNOVATION

What is USING KEY? — DuckDB's proprietary extension to `WITH RECURSIVE` that designates columns as a unique key. During recursion, if a new row has the same key as an existing row, it *replaces* the old row instead of appending alongside it.

Why It Matters for Routing: — By keying on `(from_node, to_node)`, DuckDB maintains exactly one entry per router pair. When a cheaper path is discovered, it overwrites the old entry in-place. This is exactly how a physical router's forwarding table works.

Automatic Termination: — The recursion halts when no row produces an improvement (the delta set is empty). No explicit iteration bound is needed. This mirrors the natural convergence behavior of the Bellman-Ford algorithm.

$O(V^2)$	0	Auto	None
WORKING SET SIZE	WALK EXPLOSION RISK	TERMINATION	POST-HOC AGGREGATION

STANDARD CTE VS KEYED CTE

SQLITE: STANDARD CTE

```
WITH RECURSIVE bellman(s, t, cost, hop, iter) AS (
  -- Base: direct edges
  SELECT from_node, to_node, cost, to_node, 0
  FROM edges

  UNION ALL

  -- Recursive: extend paths
  SELECT b.s, e.to_node,
         b.cost + e.cost, b.hop, b.iter + 1
  FROM bellman b
  JOIN edges e ON b.t = e.from_node
  WHERE b.s <> e.to_node
    AND b.iter < |V| - 1
)
-- POST-HOC aggregation required
SELECT s, t, MIN(cost), hop
FROM bellman GROUP BY s, t
```

DUCKDB: KEYED CTE

```
WITH RECURSIVE routing(s, t, cost, hop)
    USING KEY (s, t) AS (
  -- Base: direct edges
  SELECT from_node, to_node, cost, to_node
  FROM edges

  UNION

  -- Recursive: Bellman-Ford relaxation
  SELECT r.s, e.to_node,
         MIN(r.cost + e.cost),
         arg_min(r.hop, r.cost + e.cost)
  FROM routing r
  JOIN edges e ON r.t = e.from_node
  WHERE r.s <> e.to_node
    AND r.cost + e.cost < existing.cost
  GROUP BY r.s, e.to_node
)
-- No post-hoc aggregation needed
SELECT * FROM routing
```

Key Difference: The standard CTE accumulates all walks (UNION ALL), requiring a final GROUP BY. DuckDB's USING KEY prunes suboptimal entries during recursion (UNION), keeping only the best cost per (s, t) pair at all times.

IMPLEMENTATION: THE COMPLETE QUERY

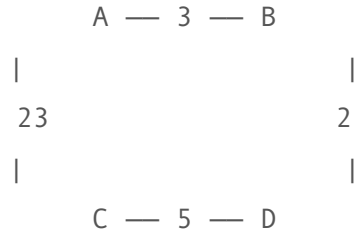
```
-- Network topology definition
CREATE TABLE edges (
    from_node TEXT,
    to_node   TEXT,
    cost      INTEGER
);

INSERT INTO edges VALUES
    ('A','B',3), ('B','A',3),
    ('A','C',23),('C','A',23),
    ('B','C',2), ('C','B',2),
    ('C','D',5), ('D','C',5);

-- Distance-Vector Routing via USING KEY
WITH RECURSIVE routing_table(
    from_node, to_node, best_cost, next_hop)
    USING KEY (from_node, to_node) AS (

    -- BASE: Direct neighbors (T=0)
    SELECT from_node, to_node,
           cost AS best_cost,
           to_node AS next_hop
    FROM edges
```

Line-by-Line Breakdown

- **USING KEY (from_node, to_node):** Defines stateful iteration; rows are updated, not appended.
- **Base Case:** Populates direct neighbor costs (T=0 discovery).
- **JOIN edges:** Simulates information propagation between neighbors.
- **recurring.routing_table:** References the accumulated FIB state.
- **WHERE ... IS NULL OR <:** Bellman-Ford relaxation condition.
- **MIN + arg_min:** Selects the lowest-cost advertisement per round.

```
      MIN(r.best_cost + e.cost),
      arg_min(r.next_hop,
              r.best_cost + e.cost)
    FROM routing_table AS r
    JOIN edges AS e
      ON r.to_node = e.from_node
    LEFT JOIN recurring.routing_table
      AS existing
      ON existing.from_node = r.from_node
     AND existing.to_node = e.to_node
    WHERE r.from_node <> e.to_node
      AND (existing.best_cost IS NULL
        OR r.best_cost + e.cost
           < existing.best_cost)
    GROUP BY r.from_node, e.to_node
)
SELECT * FROM routing_table
ORDER BY from_node, to_node;
```

Network Topology



CONVERGENCE RESULTS: FULL NETWORK

Converged Routing Table (Full Network)

FROM	TO	BEST COST	NEXT HOP
A	B	**3**	B
A	C	**5**	B
A	D	**10**	B
B	A	**3**	A
B	C	**2**	C
B	D	**7**	C
C	A	**5**	B
C	B	**2**	B
C	D	**5**	D
D	A	**10**	C
D	B	**7**	C
D	C	**5**	C

Observations

- **A to C = 5 (via B):** The algorithm discovers the indirect path A-B-C (cost 3+2=5) is cheaper than the direct link A-C (cost 23).
- **A to D = 10 (via B):** Path A-B-C-D (3+2+5=10) is optimal.
- **Convergence:** Achieved in 3 rounds for this 4-node network.

[INSERT SCREENSHOT] Streamlit Phase 2: Graph visualization showing green shortest path overlay

DISTANCE MATRIX AND CONVERGENCE EVOLUTION

[INSERT SCREENSHOT] Streamlit Phase 2: Distance Matrix D*[s][t] heatmap from the "Distance Matrix" tab

The distance matrix D*[i][j] shows the converged optimal cost from every source to every destination. Diagonal entries are zero.

[INSERT SCREENSHOT] Streamlit Phase 2: Convergence Evolution chart (FIB growth + updates per round)

The convergence chart tracks FIB size growth and the number of updates per iteration. Convergence is achieved when updates reach zero.

NETWORK FAILURE AND RE-CONVERGENCE

Simulating Link Failure

```
DELETE FROM edges
WHERE (from_node = 'B' AND to_node = 'C')
   OR (from_node = 'C' AND to_node = 'B');
```

By deleting the B-C edge and re-running the *same* recursive query, the algorithm automatically re-converges to the next-best paths.

Key Re-convergence Changes

ROUTE	BEFORE	AFTER	DELTA
A → C	5 (via B)	**23 (direct)**	+18
A → D	10 (via B)	**28 (via C)**	+18
B → C	2 (direct)	**26 (via A)**	+24
C → D	5 (direct)	5 (direct)	0

[INSERT SCREENSHOT] Streamlit Phase 4: Side-by-side before/after graph comparison + cost impact analysis bar chart

LIVE DEMO: INTERACTIVE SIMULATOR

[INSERT SCREENSHOT] Phase 1: Network Setup (topology + graph-theoretic statistics)

[INSERT SCREENSHOT] Phase 2: Convergence Simulation (routing table + path visualization)

http://localhost:8501 — Built with Streamlit + DuckDB + Pyvis + Plotly

BENCHMARK RESULTS

~10x	3	3	45+
DUCKDB SPEEDUP	BACKEND ENGINES	PYTHON ALGORITHMS	MEASUREMENTS

[INSERT SCREENSHOT] Phase 5: Execution Latency (time vs network size with 95% CI)

[INSERT SCREENSHOT] Phase 5: Speedup Bar Chart (DuckDB speedup over SQLite per size)

Finding: DuckDB's USING KEY consistently outperforms SQLite's standard CTE by 5-15x across network sizes. The gap widens with graph density due to SQLite's exponential path enumeration.

COMPLEXITY ANALYSIS

[INSERT SCREENSHOT] Phase 5: Log-log complexity plot (empirical + fitted power-law curves)

[INSERT SCREENSHOT] Appendix A: Theoretical growth curves (log-scale complexity comparison)

Algorithm Comparison

ALGORITHM	TIME	DISTRIBUTED
Bellman-Ford	$O(V^2 * E)$	Yes
Dijkstra	O(V(V+E) log V)	No
Floyd-Warshall	$O(V^3)$	No
DuckDB USING KEY	O(V * E)	N/A
SQLite Standard CTE	O(V * E * P)	N/A

P = path count: In SQLite's standard CTE, P denotes the number of distinct walks enumerated before the post-hoc GROUP BY. For dense graphs, P grows exponentially with |V|, making SQLite infeasible beyond ~12 nodes. DuckDB eliminates this factor entirely.

KEY FINDINGS

1. SQL as a PL

- SQL can fully implement distributed routing protocols.
- The declarative approach replaces hundreds of lines of imperative code with a single query.
- The recursive CTE maps directly to the iterative convergence of Bellman-Ford.

2. USING KEY Advantage

- DuckDB's USING KEY enables in-place row updates during recursion.
- Eliminates exponential path explosion inherent in standard CTEs.
- Achieves 5-15x speedup over SQLite across all tested network sizes.

3. Dynamic Resilience

- Link failure simulation via simple SQL DELETE commands.
- Re-running the same query automatically discovers next-best paths.
- No modification to the routing query is needed for re-convergence.

Central Result: Modern SQL is no longer "just for queries." With state-aware recursion, it is a first-class language for simulating, validating, and analyzing complex distributed protocols.

CONCLUSION AND FUTURE WORK

Conclusion

- Successfully demonstrated that **DuckDB's USING KEY** provides a 1:1 mapping between SQL recursion and physical router behavior.
- Built a complete **interactive evaluation suite** with 6 phases: topology setup, convergence simulation, fault injection, re-convergence, benchmarking, and algorithmic theory.
- Empirically validated that keyed CTEs maintain **polynomial scaling** while standard CTEs exhibit exponential degradation.

Future Research Directions

- **Path-Vector Protocols (BGP):** Implement BGP-like logic using SQL array types for full AS-path tracking.
- **Link-State Simulation:** Model Dijkstra/OSPF within recursive SQL for comparative benchmarking.
- **Dynamic Traffic Modeling:** Integrate real-time metrics (congestion, latency) into the edges table.
- **Large-Scale Analysis:** Test convergence on Internet-scale AS graphs to evaluate scalability limits.

END OF PRESENTATION

Thank You for Your Attention

Questions and Discussion

Mehdi Hoseyni • University of Tubingen

Database Systems Seminar — Summer Semester 2026